

5.3 DIVISÃO E CONQUISTA

A estratégia de **divisão e conquista** para a resolução de problemas - através de **decomposições e recombinações** - fornece um método poderoso e versátil para o projeto de algoritmos.

Alguns exemplos, ilustrando a grande aplicabilidade de divisão e conquista, podem ser dados por algoritmos:

- de **busca** em tabelas, como buscas seqüencial e binária;
- de **classificação**, como classificação por seleção (selectionsort), por intercalação (mergesort) e por particionamento (quicksort);
- para **multiplicação** (de matrizes e de números binários, por exemplo);
- para **seleção** (para determinar máximo e mínimo, o k^o máximo, etc.).

A estratégia opera **decompondo** um problema em subproblemas independentes, resolvendo-os e **combinando** as soluções obtidas em uma solução para o problema original.

Isso dá um processo recursivo de decomposições e recombinações.

Exemplos Introdutórios

Soma e ocorrência em lista.

Consideremos alguns problemas simples sobre listas.

- a) Desejamos achar a somatória dos elementos de uma lista de números. Podemos aplicar a idéia de **dividir para conquistar**.

Se a lista L é curta (digamos, comprimento no máximo 1), já sabemos calcular sua somatória.

Caso contrário, podemos: partir a lista L em sublistas L_1 e L_2 e aplicar recursivamente o mesmo processo até que tenhamos só sublistas curtas, das quais já sabemos tratar.

E então bastará **somar as saídas** correspondentes a essas sublistas.

A solução esboçada acima pode ser expressa pelo algoritmo recursivo

$$\begin{aligned} \text{smt}(L) &:= \text{se curta}(L) \text{ então } \text{imd}(L) \\ &\quad \text{senão } \text{soma}(\text{smt}(\text{sublist}_1(L), \text{smt}(\text{sublist}_2(L))). \end{aligned}$$

Esse algoritmo usa como **auxiliares**

- o teste unário curta (para ver se L tem comprimento no máximo 1);
- a função unária imd (dando 0 ou o único elemento da lista);
- as funções unárias sublist₁ e sublist₂ (partindo L em sublistas);
- a função binária soma (somando resultados: $\text{soma}(r_1, r_2) := r_1 + r_2$).

b) Analogamente, o problema de contar quantas vezes um objeto obj ocorre em uma lista L de objetos poderia ser resolvido por divisão e conquista da seguinte maneira.

Se a lista L é curta, basta saber se obj ocorre em L , dando saída 1, ou não, dando saída 0.

Caso contrário, partimos L em sublistas L_1 e L_2 e aplicamos recursivamente o mesmo processo até que tenhamos só sublistas curtas, das quais já sabemos tratar.

E então bastará somar as saídas correspondentes a essas sublistas.

Agora, vejamos um exemplo clássico: um algoritmo de classificação projetado através de divisão e conquista.

Classificação de listas por intercalação de sublistas.

Desejamos um algoritmo que receba como entrada uma lista e devolva essa lista classificada.

Vamos aplicar a idéia de divisão e conquista.

Se a lista de entrada é curta já está classificada.

Caso contrário a entrada será dividida em duas listas aproximadamente do mesmo tamanho a primeira metade $\text{Prim}(d)$ e a segunda metade $\text{Find}(d)$ (cf. exemplo 3.2.6 na seção 3.2 do capítulo 3).

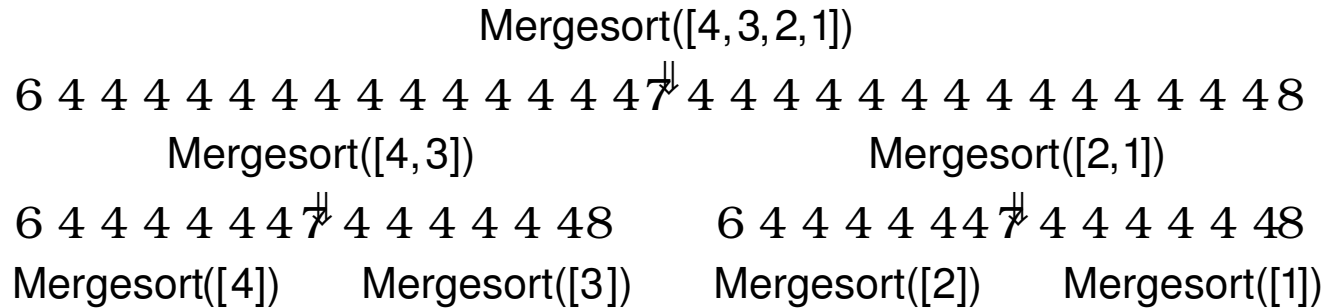
Essas duas **sublistas** serão recursivamente classificadas e intercaladas para assim se obter uma versão classificada da **lista** de entrada.

Pensando dessa maneira, podemos chegar ao algoritmo seguinte.

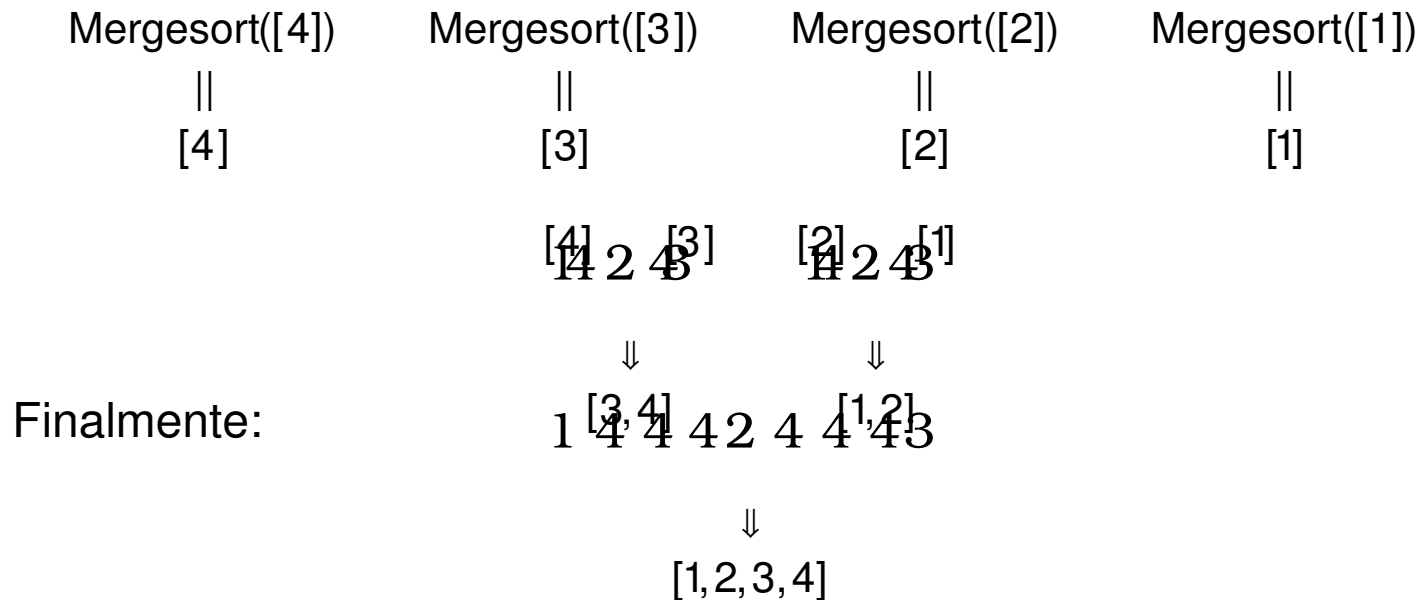
Algoritmo: Classificação de listas por intercalação de sublistas

Função Mergesort($d:D$) $\rightarrow R$ {classifica lista por intercalação de sublistas}
 { Entrada-saída: clssf (a saída r é a lista d de entrada classificada) }
 sorte $D := \text{Seq}[\text{Elmord}]$ {dados}; $R := \text{Seq}[\text{Elmord}]$ {resultados};
 var $d_1, d_2:D$; $r_1, r_2:R$; $r:R$ {saída}; $n:IN$ {comprimento da lista};
 0. $n \leftarrow \text{compr}(d)$; {comprimento da entrada}
 1. se $n \leq 1$ {testa se a entrada é curta ou não: 12}
 2. então { $n \leq 1$ (saída direta): 5}
 3. $r \leftarrow d$; {entrada curta já está classificada}
 4. retorne-saída(r); {dá saída direta d }
 5. fim-então {2: entrada curta (saída direta)}
 6. senão { $n > 1$ (recursão): 11}
 7. $d_1 \leftarrow \text{Prim}(d)$; $d_2 \leftarrow \text{Fin}(d)$; {parte em duas metades}
 8. $r_1 \leftarrow \text{Mergesort}(d_1)$; $r_2 \leftarrow \text{Mergesort}(d_2)$; {recursão}
 9. $r \leftarrow \text{Intercl}(r_1, r_2)$; {intercala partes classificadas}
 10. retorne-saída(r); {dá saída intercalada r }
 11. fim-senão {6: entrada não curta (recursão)}
 12. fim-se {1: $n \leq 1$?}
 13. fim-Função {fim do algoritmo Mergesort: classificação por intercalação}

A execução desse algoritmo **Mergesort** sobre a entrada $d := [4, 3, 2, 1]$, temos



As folhas têm chamadas sobre sublistas curtas, que já estão classificadas.



A idéia básica da divisão e conquista é decompor um **problema** em **subproblemas menores** independentes, resolvê-los e combinar as soluções obtidas em uma solução para o problema original.

Esse **processo recursivo** funciona da seguinte maneira:

- dada uma entrada, se ela é suficientemente simples, obtemos diretamente uma saída correspondente;
- caso contrário, ela é decomposta em entradas mais simples, para as quais aplicamos o mesmo processo, obtendo saídas correspondentes, que são então combinadas em uma saída para a entrada original.

De acordo com a **estratégia**, cada entrada será decomposta em, digamos, duas partes, cujas saídas serão depois combinadas, fornecendo uma saída para a entrada original. Para isso, usamos procedimentos (funções) como se segue:

- duas funções unárias $part_1$ e $part_2$ para decompor entradas;
- uma função binária $cmbn_2$ para combinar saídas;
- um procedimento unário $smpl$ para testar se a entrada é simples ou não; e
- uma função unária drt dando saídas para entradas simples.

Formulação da divisão e conquista binária

Função: $\text{Div_Conq_2} (d:D) \rightarrow R$ {Divisão e Conquista binária}
 {Sortes D (entrada); R (resultados)} com a seguinte **formulação recursiva**.

$$\text{Div_Conq_2}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{smp}(d) \\ \text{combn_2}(\text{Div_Conq_2}(\text{part}_1(d), \text{Div_Conq_2}(\text{part}_2(d))) & \text{caso contrário} \end{cases}$$

Instâncias de divisão e conquista binária.

Procedimento smt

	Div_Conq_2	smt
Sortes		
entrada	$d:D$	L : Lista de números
saída	R	Número
Teste		
simples	$\text{smp}(d)$	$\text{curta}(L)$
Funções		
direto	$\text{drt}(d)$	$\text{imd}(L)$
primeira parte	$\text{part}_1(d)$	$\text{sublist}_1(L)$
segunda parte	$\text{part}_2(d)$	$\text{sublist}_2(L)$
combinação	$\text{combn_2}(r_1, r_2)$	$\text{soma}(r_1, r_2)$

Algoritmo: MergesortMergesort(d) := Div_Conq_2(d)onde $D := \text{Seq}[\text{Elmord}]; R := \text{Seq}[\text{Elmord}]$ $\text{simpl}(d) := \text{compr}(d) \leq 1$ $\text{drt}(d) := d$ $\text{part}_1(d) := \text{Prim}(d)$ $\text{part}_2(d) := \text{Fin}(d)$ $\text{cmbn_2}(r_1, r_2) := \text{Intercl}(r_1, r_2)$

{invoca Div_Conq_2}

{definições}

{sortes};

{teste simpl};

{operação drt};

{primeira parte};

{segunda parte};

{combinação}.

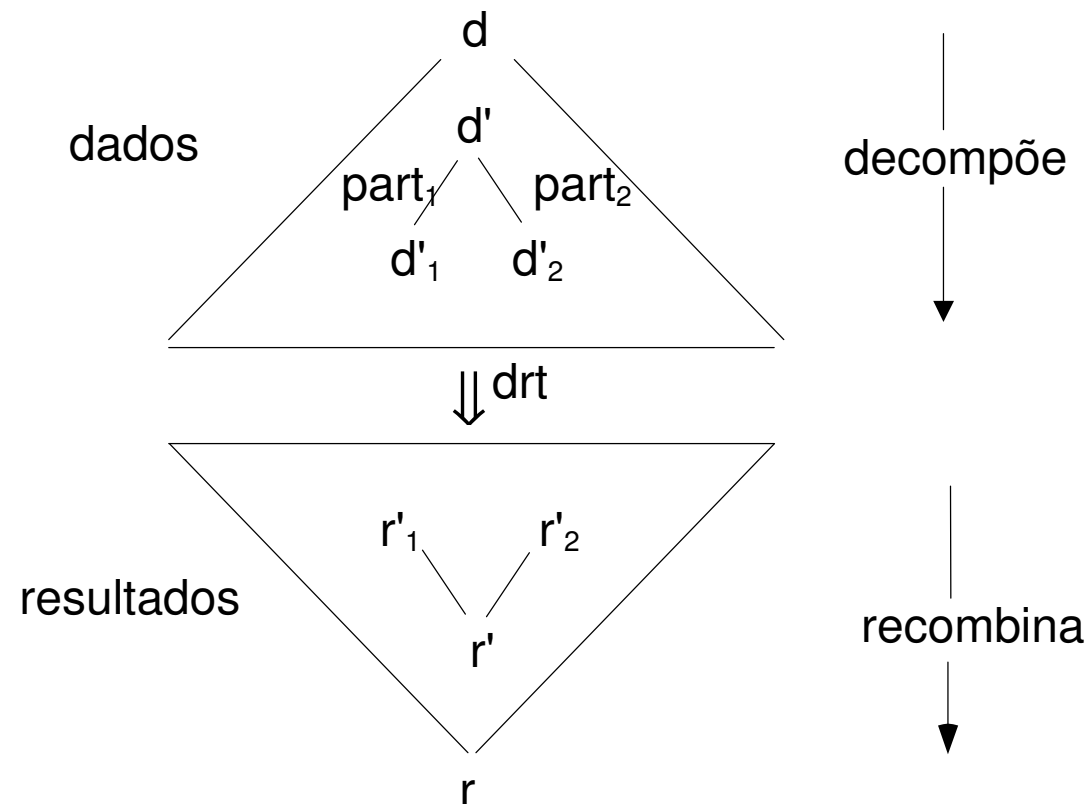
Outros exemplos de algoritmos de divisão e conquista binária são algoritmos:

- de classificação por seleção "selectionsort";
- para determinar máximo e mínimo de tabela.

Algoritmos de divisão e conquista em geral são recursivos. A **recursão** desempenha importante papel na divisão e conquista.

O funcionamento da recursão na divisão e conquista pode ser visualizado considerando-se as árvores que o procedimento abstrato Div_Conq_2 (constrói, em três estágios.

1. **árvore de dados**: é construída da raiz em direção à folhas (por decomposições repetidas das instâncias de dados) até que as folhas sejam simples.
2. aplica-se drt para transferir as folhas do sorte de dados para o de resultados.
3. **árvore de resultados**: é construída das folhas em direção à raiz.



Agora, a formulação binária anterior se generaliza para sua **versão m-ária**.

Função: $\text{Div_Conq_m} (d:D) \rightarrow R$ {Divisão e Conquista m-ária}
 {Sortes D (de entrada); R (de saída)} com a seguinte formulação recursiva

$$\text{Div_Conq_m}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{smpl}(d) \\ \text{combn_m}(\text{Div_Conq_m}(\text{part}_1(d), K, \text{Div_Conq_m}(\text{part}_m(d)))) & \text{caso contrário} \end{cases}$$

Uma formulação mais liberal contempla o caso em que a combinação pode utilizar, além dos resultados das partes, também a entrada que foi decomposta.

Neste caso a operação de combinação é **(m+1)-ária**.

Um exemplo imediato de divisão e conquista ternária ($m=3$) pode ser obtido, modificando-se o algoritmo Quicksort (cf. seção 4.1 do capítulo 4).

Essa nova versão dividiria a entrada em três por comparação com o **pivô**:

- a primeira com os elementos menores do que o pivô,
- a segunda com os elementos iguais ao pivô, e
- a terceira com os elementos maiores do que o pivô. Sua combinação seria a concatenação.

Ainda outro exemplo de divisão e conquista não binária é o **algoritmo de Strassen** para multiplicação de matrizes quadradas.

Para multiplicar duas matrizes $n \times n$, esse algoritmo decompõe cada uma delas em 4 blocos $(n/2) \times (n/2)$ e calcula 7 produtos de matrizes, dos quais os 4 blocos da matriz produto podem ser obtidos.

Examinemos agora o caso de **divisão e conquista unária ($m=1$)**.

Função: $\text{Div_Conq_1} (d:D) \rightarrow R$ {Divisão e Conquista unária}
 {Sortes D {dados de entrada}; R (resultados de saída)}

com a seguinte formulação recursiva

$$\text{Div_Conq_1}(d) = \begin{cases} \text{drt}(d) & \text{se } \text{smpl}(d) \\ \text{combn_1}(\text{Div_Conq_1}(\text{part}_1(d))) & \text{caso contrário} \end{cases}$$

No caso de divisão e conquista unária ($m=1$) o termo 'divisão' não parece apropriado, 'redução' sendo talvez mais adequado.

Projeto de algoritmos por divisão e conquista

A fim de aplicar a estratégia de divisão e conquista, devemos formular os **procedimentos auxiliares** de maneira apropriada. Problemas de classificação podem ser usados para ilustrar essa idéia.

Divisão e conquista para classificação.

Consideremos algoritmos de classificação, cujas entradas e saídas consistem de seqüências de elementos (de um conjunto ordenado).

Podemos definir o teste smpl e a função unária drt , sobre dados, por

$$\text{smpl}(d) \text{ sse } \text{compr}(d) \leq 1 \text{ e } \text{drt}(d) := d.$$

Duas possíveis definições para decomposições e recombinações são as seguintes.

1. Podemos definir $\text{part}_1(d)$ e $\text{part}_2(d)$ como a primeira e segunda metades de d e $\text{cmbn_2}(r_1, r_2)$ como a intercalação de r_1 com r_2 .
2. Também podemos definir $\text{part}_1(d)$ como a seqüência consistindo do valor mínimo em d , $\text{part}_2(d)$ como o resultado de remover esse valor mínimo de d , e $\text{cmbn_2}(r_1, r_2)$ como a concatenação de r_1 seguida por r_2 .

Essas **duas** maneiras de implementar dão origem a algoritmos de classificação familiares:

- por intercalação (mergesort) no caso 1, e
- seleção (straight selectionsort) no caso 2.

Em ambos os casos cada entrada é decomposta em duas menores, mas com distintas relações de tamanho.

Essa distinção entre as duas relações de tamanho influencia a **altura da árvore** de execução.

Considerando a entrada $d := [1, 2, 3, 4]$, teremos as seguintes árvores de dados:

1. na classificação por intercalação (mergesort)

$$\begin{array}{c} [1,2,3,4] \\ \hline [1,2] \quad [3,4] \\ \hline [1] \quad [2] \quad [3] \quad [4] \end{array}$$

2. na classificação por seleção (straight selectionsort)

$$\begin{array}{c} [1,2,3,4] \\ \hline [1] \quad [2,3,4] \\ \quad \quad \hline [2] \quad [3,4] \\ \quad \quad \quad \hline [3] \quad [4] \end{array}$$

O exemplo ilustra uma idéia importante na estratégia de divisão e conquista de que entrada em partes (aproximadamente) iguais leva a árvores menores.

O chamado *princípio de equipartição* aconselha partir a entrada em partes (aproximadamente) iguais a fim de diminuir a profundidade das árvores, assim tendendo a melhorar o desempenho do algoritmo resultante.

Máximo e mínimo de tabela por divisão e conquista

Algoritmo: Máximo e mínimo de tabela (divisão e conquista binária)

Função Max_min(Tab:D) $\rightarrow$ R {determina máximo e mínimo de tabela}

{ Saída: valores r_M :Elmord(máximo) e r_m :Elmord (mínimo) da entrada}
sorte D := vetor [p..q] de Elmord {dados}; R := Par[Elmord] {resultados};
var {auxiliares} p_s, q_i : IN {índices de subtabelas}; s_M, s_m, t_M, t_m : Elmord
 {máximo e mínimo de partes}; n, k : IN {tamanhos de tabela};

```

0.  $n \leftarrow q - p + 1$ ;                                {tamanho da tabela}
1. se  $n \leq 2$                                          {testa se a tabela é pequena ou não: 23}
2. então                                             { $n \leq 2$  (saída direta): 14}
3.     se  $n = 1$                                        { $n = 1$ ?: 12}
4.         então                                       { $n = 1$ : 7}
5.              $r_M \leftarrow \text{Tab}[p]$ ;                {máximo}
  
```

6.	$r_m \leftarrow \text{Tab}[p] ;$	{mínimo}
7.	<u>fim-então</u>	{4: n=1}
8.	<u>senão</u>	{n=2:11}
9.	$r_M \leftarrow \text{Máx}(\text{Tab}[p], \text{Tab}[q]) ;$	{máximo}
10.	$r_m \leftarrow \text{mín}(\text{Tab}[p], \text{Tab}[q]) ;$	{mínimo}
11.	<u>fim-senão</u>	{8: n=2}
12.	<u>fim-se</u>	{3: n=1?}
13.	<u>retorne-saída</u> (r_M, r_m) ;	{dá saída direta}
14.	<u>fim-então</u>	{2: tabela pequena (saída direta)}
15.	<u>senão</u>	{n>2 (recursão): 22}
16.	$k \leftarrow \lceil n/2 \rceil ; q_i \leftarrow p+k ; p_s \leftarrow q_i - 1 ;$	{duas metades}
17.	$(s_M, s_m) \leftarrow \text{Max_min}(\text{Tab}[p..p_s]) ;$	{recursão: 1ª metade}
18.	$(t_M, t_m) \leftarrow \text{Max_min}(\text{Tab}[q_i..q]) ;$	{recursão: 2ª metade}
19.	$r_M \leftarrow \text{Máx}(s_M, t_M) ;$	{combinação: máximo}
20.	$r_m \leftarrow \text{mín}(s_m, t_m) ;$	{combinação: mínimo}
21.	<u>retorne-saída</u> (r_M, r_m) ;	{dá saída combinada}
22.	<u>fim-senão</u>	{15: tabela não pequena (recursão)}
23.	<u>fim-se</u>	{1: n≤2?}
24.	<u>fim-Função</u>	{fim do algoritmo Max_min: máximo e mínimo de tabela}

E algoritmo **Max_min** é uma instância da formulação acima da divisão e conquista binária Div_Conq_2.

Examinemos agora exemplos de algoritmos projetados por divisão e conquista unária.

Ocorrência de chave em tabela por divisão e conquista.

Este algoritmo determina se uma **chave** ocorre ou não uma tabela, armazenada como vetor tab, por meio de divisão e conquista unária.

Caso o elemento procurado ainda não tenha sido encontrado, a busca original é reduzida a uma numa tabela com um elemento a menos.

Algoritmo: Ocorrência em tabela (por divisão e conquista unária)

Função Oc_seq(ch:Elm, tab:Cnj) $\rightarrow$ Bool {determina se chave ocorre em tabela}

{Entrada: par consistindo de chave ch:Elm e tabela tab:Cnj
Saída: V (se ch ocorre em tab) e F (se ch não ocorre em tab)}

sorte Cnj := vetor[p..q] de Elm;

```

    var {auxiliares} achou:Bool; n:IN {tamanho de tabela};
0. n ← q - p + 1;                                {tamanho da tabela}
1. se n=1                                         {testa se a tabela é pequena ou não: 17}
2. então                                         {n=1 (saída direta): 7}
3.     se tab[p]=ch                               {tab[p]=ch?: 6}
4.         então achou ← V ;                       {tab[p]=ch}
5.         senão achou ← F ;                       {tab[p]≠ch}
6.     fim-se                                     {3: tab[p]=ch?}
7.     retorne-saída(achou) ;                     {dá saída direta}
8. fim-então                                     {2: tabela pequena (saída direta)}
9. senão                                         {n>1 (recursão): 16}
10.    se tab[p]=ch                               {tab[p]=ch?: 13}
11.        então q ← p ;                           {tab[p]=ch}
12.        senão p ← p + 1 ;                       {tab[p]≠ch}
13.    fim-se                                     {9: tab[p]=ch?}
14.    achou ← Oc_seq(ch:Elm, tab:Cnj) ;           {recursão: parte}
15.    retorne-saída(achou) ;                     {retorna saída}
16. fim-senão                                   {9: tabela não pequena (recursão)}
17. fim-se                                       {1: n=1?}
18. fim-Função      {fim do algoritmo Oc_seq: ocorrência em tabela (seqüencial)}

```

Este algoritmo simula uma busca seqüencial, reduzindo a busca original a uma numa tabela com um elemento a menos, caso o elemento procurado ainda não tenha sido encontrado.

Esse algoritmo foi obtido especializando-se a formulação **Div_Conq_1 da divisão e conquista unária**. Podemos identificar a seguinte estrutura.

Oc_seq(d) := Div_Conq_1 (d)	{invoca Div_Conq_1}
<u>onde</u>	{definições}
D := Par[Elmx Cnj]; R := Bool	
<u>com</u> Cnj := <u>vetor</u> [p..q] <u>de</u> Elm	{sortes};
smp1(ch, tab) := compr(tab) = 1	{teste smp1};
drt(ch, tab) := linhas de 2 a 6	{operação drt};
part ₁ (ch, tab) := linhas de 10 a 13	{parte};
cmbn ₁ (r ₁) := r ₁	{combinação}.

Essa estrutura exhibe o algoritmo Oc_seq como uma instância da formulação da divisão e conquista unária.

Examinemos um exemplo de algoritmo projetados por divisão e conquista liberal.

Consideremos o problema de determinar o mínimo de uma tabela, um problema análogo ao visto na seção 2.4 do capítulo 2 (cf. algoritmo 2.1 no exemplo 2.4.1).

Mínimo de tabela por divisão e conquista.

Algoritmo: Mínimo de tabela (por divisão e conquista unária liberal)

Função El_mín(tab:D) → R {determina mínimo de tabela}
 { Entrada: tabela tab:D representando elementos (de conjunto ordenado)
 Saída: valor r :Elmord (mínimo da tabela tab:D de entrada)}
 sorte D := vetor [p..q] de Elmord {dados}; R := Elmord {resultados};
 var {auxiliares} d₁:D {parte de tabela}; r, r₁:Elmord {mínimo (de parte)};
 n:IN {tamanho de tabela};

0. n ← q - p + 1; 1. <u>se</u> n ≤ 1 2. <u>então</u> <u>retorne-saída</u> (tab[p]); 3. <u>senão</u> 4. d ₁ ← tab[p + 1..q]; 5. r ₁ ← El_mín(d ₁); 6. r ← mín(tab[p], r ₁); 7. <u>retorne-saída</u> (r);	{tamanho da tabela} {testa se a tabela é pequena ou não: 9} {dá saída direta} {n > 1 (recursão): 8} {parte} {recursão: parte} {combinação (liberal)} {retorna saída}
--	---

- | | |
|-----------------------|---|
| 8. <u>fim-senão</u> | {3: tabela não pequena (recursão)} |
| 9. <u>fim-se</u> | {1: $n=1?$ } |
| 10. <u>fim-Função</u> | {fim do algoritmo El_mín: mínimo de tabela} |

O algoritmo Mínimo de tabela foi obtido especializando-se a formulação Div_Conq_1L da **divisão e conquista unária liberal**.

El_mín(tab) := Div_Conq_1L(tab)	{invoca Div_Conq_1L}
<u>onde</u>	{definições}
D := <u>vetor</u> [p..q] <u>de</u> Elmord; R := Elmord	{sortes};
smp1(tab) := compr(tab) ≤ 1	{teste smp1};
drt(tab) := tab[p]	{operação drt};
part ₁ (tab) := tab[p+1..q]	{parte};
cmbn_1L(r ₁ , tab) := mín(tab[p], r ₁)	{combinação liberal}.

Um exemplo interessante de divisão e conquista ternária ($m=3$): um algoritmo para multiplicar números inteiros em uma base (e. g. em base 2).

Produto de números binários por divisão e conquista.

Calcula o produto de dois números binários p e q , com n bits, escrevendo-os na forma $p = p_1 * 2^{(n/2)} + p_2$ e $q = q_1 * 2^{(n/2)} + q_2$, onde p_1, p_2, q_1 e q_2 têm $n/2$ bits cada um.

Algoritmo: Produto de dois números binários (divisão e conquista ternária)

Função Prod_bin($p, q: \text{Bin}$) $\rightarrow \text{Bin}$ {calcula o produto de 2 números binários}
 { Entrada: números binários $p, q: \text{Bin}$ (na forma $p = p_1 * 2^{(n/2)} + p_2$ e $q = q_1 * 2^{(n/2)} + q_2$)
 Saída: valor $r: \text{Bin}$ (produto dos números binários $p, q: \text{Bin}$ de entrada) }
 var {auxiliares} $x, y, z, t, u, v: \text{Bin}; n: \mathbb{N}$ {números de bits};

1. se $n=1$ {testa se os números são pequenos ou não: 15}
2. então { $n=1$ (saída direta): 5}
3. $r \leftarrow p.q;$ {multiplica seus (únicos) bits}
4. retorne-saída(r) {dá saída direta}
5. fim-então {2: números pequenos (saída direta)}
6. senão { $n > 1$ (recursão): 14}
7. $x \leftarrow p_1 + p_2; y \leftarrow q_1 + q_2;$ {soma números com $n/2$ bits}
8. $z \leftarrow \text{Prod_bin}(x_2, y_2);$ {recursão com $n/2$ bits}
9. $t \leftarrow (x_1 * y_1) * 2^n + (x_1 * y_2 + x_2 * y_1) * 2^{(n/2)} + z;$
10. $u \leftarrow \text{Prod_bin}(p_1, q_1);$ {recursão com $n/2$ bits}
11. $v \leftarrow \text{Prod_bin}(p_2, q_2);$ {recursão com $n/2$ bits}
12. $r \leftarrow u * 2^n + (t - u - v) * 2^{(n/2)} + v;$ {combinação}
13. retorne-saída(r) {dá saída combinada}
14. fim-senão {6: números não pequenos (recursão)}

15. fim-se {1: n=1?}
16. fim-Função {fim do algoritmo Prod_bin: produto de números binários}

O algoritmo Prod_bin é uma instância da formulação acima da divisão e conquista ternária Div_Conq_3 (com $m:=3$).

Análise da complexidade de algoritmos de divisão e conquista

Agora vamos ver como se pode efetuar a **análise de complexidade pessimista** de alguns algoritmos típicos desenvolvidos pela estratégia de divisão e conquista.

Em cada caso, faremos inicialmente uma **estimativa** da complexidade, passando a seguir a efetuar uma análise mais cuidadosa e detalhada

Algoritmos desenvolvidos por divisão e conquista geralmente são recursivos, tendo análise menos simples do que no caso de algoritmos iterativos.

Aqui, uma questão **fundamental** é a determinação do número de chamadas recursivas (similar à questão do número de iterações quando indefinidas).

Complexidade de ocorrência em tabela.

Consideremos o algoritmo Oc_seq (para determinar se uma chave ocorre numa tabela por busca seqüencial)

Esse **algoritmo Oc_seq** foi obtido especializando-se a formulação Div_Conq_1 da divisão e conquista unária.

Podemos estimar a complexidade pessimista do algoritmo Oc_seq da seguinte maneira:

- a cada invocação recursiva o tamanho da tabela diminui de 1, assim serão feitas n chamadas recursivas para uma tabela de tamanho n;
- no caso simples (linhas de 2 a 8), é feito um acesso à tabela (linha 3);
- no caso não simples (linhas de 9 a 16), é feito um acesso à tabela (linha 10).

Portanto, a **complexidade do algoritmo Oc_seq** tem ordem

$$c_p^{\leq} [\text{Oc_seq}](n) = \Theta(n).$$

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo `Oc_seq` recebe como entrada d um par ordenado (registro), consistindo de:

uma chave $ch:Elm$ e uma tabela $tab:\underline{vetor}[p..q]$ de Elm .

Como é natural, para tamanho $tam(d)$ da entrada d é tomada a dimensão $compr(tab)$ da tabela tab , calculado no algoritmo `Oc_seq` pela linha

0. $n \leftarrow q - p + 1$; {tamanho da tabela}

e seu teste de simplicidade é se $n=1$ (linha 1).

Vemos que o teste de simplicidade do algoritmo `Oc_seq` é caracterizado por

$simpl(d)$ sse $tam(d) \leq 1$ (cf linha 1 se $n=1$).

Afim de determinar quantas invocações recursivas são feitas, ou seja quantas vezes a **entrada d** deve ser partida (por aplicação de $part_1$) até que o teste $simpl$ seja satisfeito, examinemos o trecho das linhas 10 a 13:

10. <u>se</u> $tab[p]=ch$	{ $tab[p]=ch?$: 13}
11. <u>então</u> $q \leftarrow p$	{ $tam[part_1(d)]=q-q+1$ }
12. <u>senão</u> $p \leftarrow p+1$	{ $tam[part_1(d)]=q-(p+1)+1$ }
13. <u>fim-se</u>	{9: $tab[p]=ch?$ }

Assim, temos $tam[part_1(d)] = 1$ na linha 11 e $tam[part_1(d)] = q - p$ na linha 12. Logo:

$$tam[part_1(d)] \leq tam(d) - 1.$$

Pelo visto acima, os dados dessa seqüência têm tamanhos como se segue

dados	d_0	d_1	$\dots$	d_j	$\dots$	$d_{H(d)}$
tamanhos	n_0	$n_0 - 1$	$\dots$	$n_0 - j$	$\dots$	$n_0 - H(d)$

Quanto vale $H(d_0)$?

A **altura da árvore** de execução: a primeira vez que se consegue satisfazer o teste `smpl`.

$$H(d_0) = \min\{j \in \mathbb{N} / n_0 - j \leq 1\}.$$

Portanto, a árvore de execução tem altura $H(d_0) \leq n_0 - 1$, ou seja

$$H(d) \leq tam(d) - 1.$$

Prosseguindo de modo similar ao caso da iteração indefinida (cf. 3.2.5 na seção 3.2 do capítulo 3), o desempenho do algoritmo `Oc_seq` sobre entrada d é:

- Se a entrada d satisfaz o teste $\text{smpl}(d)$ (i. e. $\text{tam}(d) \leq 1$):

$$\text{desemp}[\text{Oc_seq}](d) = \text{ava}[\text{smpl}](d) + \text{desemp}[\text{drt}](d).$$

- Se a entrada d não satisfaz o teste $\text{smpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{Oc_seq}](d) = & \text{ava}[\text{smpl}](d) + \text{desemp}[\text{part}_1](d) + \\ & + \text{desemp}[\text{Oc_seq}](\text{part}_1(d)) + \\ & + \text{desemp}[\text{cmbn_1}](\text{Oc_seq}(\text{part}_1(d))). \end{aligned}$$

Nesse caso, é natural tomar como **operação fundamental** a comparação $\text{tab}[i] = \text{ch}$ de elemento acessado na tabela com a chave ch . Assim, temos

Processo	linhas	comparações
smpl	0 a 1	0
drt	2 a 6	1
part_1	10 a 13	1
cmbn_1	15	0

Então, em termos de número de comparações, obtemos:

- para a entrada simples (com $tam(d) \leq 1$), $desemp[Oc_seq](d)$ tem as contribuições

$$\begin{array}{rcl} ava[smpl](d) & = & 0 \\ + & & + \\ desemp[drt](d) & = & 1 \end{array}$$

i. e. $desemp[Oc_seq](d) = 1$;

- para a entrada não simples (com $tam(d) > 1$), $desemp[Oc_seq](d)$ tem as contribuições

$$\begin{array}{rcl} ava[smpl](d) & = & 0 \\ + & & + \\ desemp[part_1](d) & = & 1 \\ + & & + \\ desemp[Oc_seq](part_1(d)) & = & desemp[Oc_seq](part_1(d)) \\ + & & + \\ desemp[cmbn_1](Oc_seq(part_1(d))) & = & 0 \end{array}$$

i. e. $desemp[Oc_seq](d) = 1 + desemp[Oc_seq](part_1(d))$.

Assim, o desempenho do algoritmo Oc_seq sobre entrada d é descrito pela recorrência;

$$desemp[Oc_seq](d) = \begin{cases} 0 & \text{se } tam(d) \leq 1 \\ desmp[Oc_seq](part_1(d)) + 1 & \text{se } tam(d) > 1 \end{cases}$$

E o desempenho do algoritmo Oc_seq sobre entrada d é dado pela somatória

$$\frac{\begin{array}{c} \text{rec.} \\ 1 \quad + \quad \dots \quad + \quad 1 \end{array}}{H(d)} + \frac{\begin{array}{c} \text{fim} \\ 1 \end{array}}$$

Assim, obtemos a seguinte cota superior para o **desempenho do algoritmo Oc_seq**

$$desemp[Oc_seq](d) = H(d) + 1 \leq tam(d).$$

O pior caso, para entradas com tamanho até n , vem a ser

$$h_{\leq}(n) := \text{Máx}\{H(d) \in \mathbb{N} / tam(d) \leq n\}.$$

Agora, a complexidade pessimista do algoritmo Oc_seq vem a ser

$$c_p^{\leq}[Oc_seq](n) := \text{Máx}\{desemp[Oc_seq](d) / tam(d) \leq n\};$$

Logo, é cota superior para a complexidade pessimista de Oc_seq

$$c_p^{\leq} [\text{Oc_seq}](n) \leq n.$$

Portanto, a **complexidade pessimista do algoritmo Oc_seq** tem ordem

$$c_p^{\leq} [\text{Oc_seq}](n) = \Theta(n).$$

Essa análise mostra que a complexidade pessimista do algoritmo Oc_seq, em termos de número de comparações com a chave, tem ordem linear no tamanho da entrada.

Examinemos agora um caso de divisão e conquista unária liberal.

Complexidade de mínimo de tabela.

Consideremos o **algoritmo El_mín** (para determinar o elemento mínimo de um conjunto ordenado, armazenado como vetor tab por busca seqüencial).

Esse algoritmo El_mín foi obtido especializando-se a formulação Div_Conq_1L da divisão e conquista unária liberal.

Podemos estimar a complexidade pessimista do algoritmo El_mín como se segue.

- A cada invocação recursiva o tamanho da tabela diminui de 1, assim serão feitas n chamadas recursivas para uma tabela de tamanho n .
- O caso simples (linha 2) não envolve nenhuma comparação.
- No caso não simples (linhas de 3 a 8) envolve uma comparação (linha 6).

Portanto, a **complexidade do algoritmo El_mín** tem ordem

$$c_p^{\leq}[\text{El_mín}](n) = \Theta(n).$$

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo El_mín recebe como entrada d uma tabela $\text{tab} : \text{vetor}[p..q]$ de Elmord.

para $\text{tam}(d)$ é tomada a dimensão $\text{compr}(\text{tab})$ da tabela tab .

Esse tamanho é calculado na linha 0 do algoritmo El_mín e seu teste de simplicidade é $\text{se } n \leq 1$ (linha 1), logo

$$\text{simpl}(d) \text{ sse } \text{tam}(d) \leq 1.$$

Queremos saber quantas invocações recursivas são feitas até que o teste simpl seja satisfeito.

No algoritmo *El_mín* a operação de partir é

$$\text{part}_1(\text{tab}) := \text{tab}[p+1..q] \text{ (linha 4).}$$

Assim, temos $\text{tam}[\text{part}_1(d)] = q - (p+1) + 1$. Logo

$$\text{tam}[\text{part}_1(d)] = \text{tam}(d) - 1.$$

O algoritmo *El_mín* aplicado a uma entrada d fará uma seqüência de chamadas recursivas até que o teste *smpl* seja satisfeito, i. e. até obter

$$d_{H(d)} \text{ com } \text{tam}[d_{H(d)}] \leq 1.$$

Mas, $\text{tam}(d_j) = \text{tam}(d) - j$.

A altura $H(d_0)$ da árvore de execução é

$$H(d_0) = \text{mín} \{j \in \mathbb{N} / \text{tam}(d_0) - j \leq 1\}.$$

Portanto, a **árvore de execução** tem altura $H(d) = \text{tam}(d) - 1$.

Com $H(d)$ o desempenho do algoritmo *El_mín* sobre entrada d é como se segue:

- Se a entrada d satisfaz o teste *smpl*(d) (i. e. $\text{tam}(d) \leq 1$):

$$\text{desemp}[\text{El_mín}](d) = \text{ava}[\text{smpl}](d) + \text{desemp}[\text{drt}](d).$$

- Se a entrada d não satisfaz o teste $\text{simpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{El_mín}](d) = & \text{aval}[\text{simpl}](d) + \text{desemp}[\text{part}_1](d) + \\ & + \text{desemp}[\text{El_mín}](\text{part}_1(d)) + \\ & + \text{desemp}[\text{cmbn_1L}](d, \text{El_mín}(\text{part}_1(d))). \end{aligned}$$

O **natural** é tomar como operação fundamental $\text{mín}(r, \text{tab}[i])$ que compara um elemento r de Elmord com o elemento $\text{tab}[i]$ acessado na tabela.

Mas, só $\text{cmbn_1L}(r_1, \text{tab}) := \text{mín}(\text{tab}[p], r_1)$ usa essa operação (uma vez).

Então, em termos de número de comparações, o desempenho do algoritmo El_mín sobre entrada d é descrito pela recorrência

$$\text{desemp}[\text{El_mín}](d) = \begin{cases} 0 & \text{se } \text{tam}(d) \leq 1 \\ \text{desemp}[\text{El_mín}](\text{part}_1(d)) + 1 & \text{se } \text{tam}(d) > 1 \end{cases}$$

Portanto, o **desempenho** do algoritmo El_mín sobre entrada d é dado por

$$\frac{1 \quad + \quad \dots \quad + \quad 1}{H(d)}$$

Assim, obtemos

$$desemp[El_mín](d) = H(d) = tam(d).$$

Como antes, vemos que o **pior caso** da profundidade de recursão do algoritmo `El_mín` sobre entradas com tamanho até n , é dado por

$$h_{\leq}(n) = n; \text{ logo } h_{\leq}(n) = \Theta(n).$$

Portanto, obtemos $c_p^{\leq}[El_mín](n) = n$, logo

$$c_p^{\leq}[El_mín](n) = \Theta(n).$$

Complexidade de classificação de listas por intercalação.

Consideremos o algoritmo Mergesort(para classificar listas por intercalação de sublistas), obtido especializando-se a formulação `Div_Conq_2` da divisão e conquista binária.

Estimativa da **complexidade pessimista do algoritmo Mergesort**:

- A cada invocação recursiva a lista é dividida aproximadamente ao meio, assim serão feitas $\Theta(\log_2 n)$ chamadas recursivas para uma tabela de tamanho n .
- Só $r \leftarrow \text{Intercl}(r_1, r_2)$ (linha 9) envolve comparações: $O(n)$.

Assim, podemos ver que $c_p^{\leq} [\text{Mergesort}](n) = O(n \cdot \log_2 n)$.

A seguir, vamos fazer uma análise mais cuidadosa e detalhada.

O algoritmo Mergesort recebe como entrada uma lista $d: \text{Seq}[\text{Elmord}]$.

para $\text{tam}(d)$ tomemos o comprimento $\text{compr}(d)$ da lista d .

Esse comprimento é determinado na linha 0 e seu teste de simplicidade é

se $n \leq 1$ (linha 1), e o teste de simplicidade $\text{smpl}(d)$ sse $\text{tam}(d) \leq 1$.

Agora surge a questão de quantas invocações recursivas são feitas, ou seja quantas vezes a **entrada d** deve ser partida até que o teste smpl seja satisfeito.

As operações part_1 e part_2 do algoritmo Mergesort são efetuadas pela linha 7:

$$\text{part}_1(d) := \text{Prim}(d) \qquad \{\text{compr}[\text{Prim}(d)] = \lfloor \text{compr}(d)/2 \rfloor\}$$

$$\text{part}_2(d) := \text{Fin}(d) \qquad \{\text{compr}[\text{Fin}(d)] = \lceil \text{compr}(d)/2 \rceil\}.$$

Assim, temos $\text{tam}[\text{part}_1(d)] = \lfloor \text{tam}(d)/2 \rfloor$ e $\text{tam}[\text{part}_2(d)] = \lceil \text{tam}(d)/2 \rceil$.

Consideremos o tamanho máximo das decomposições de entrada d (não simples):

$$S(d) := \text{Máx}\{\text{tam}[\text{part}_1(d)], \text{tam}[\text{part}_2(d)]\}.$$

Obtemos $S(d) = \lceil \text{tam}(d)/2 \rceil$.

O algoritmo **Mergesort** aplicado a uma entrada d_0 fará uma seqüência de chamadas recursivas gerando (por aplicação de part_1 e part_2) uma árvore de dados até que o teste smpl seja satisfeito, ou seja até obter $d_{H(d)}$ com $\text{tam}[d_{H(d)}] \leq 1$.

Os níveis dessa árvore têm tamanhos máximos como se segue:

nível d_i	d_0	d_1	...	d_j	...	$d_{H(d)}$
tamanho $S(d_j)$	n_0	$\lceil n_0/2 \rceil$	...	$\lceil n_0/(2^j) \rceil$	...	$\lceil n_0/(2^{H(d)}) \rceil$

Quanto vale $H(d_0)$? Trata-se da altura da árvore de execução: a primeira vez em que todas as folhas satisfazem o teste *smpl*. Assim, temos

$$H(d_0) = \min\{j \in \mathbb{N} / \lceil \text{tam}(d_0)/(2^j) \rceil \leq 1\}.$$

A árvore de execução tem altura $H(d_0) = \lceil \log_2 n_0 \rceil$, ou seja

$$H(d) = \lceil \log_2 [\text{tam}(d)] \rceil.$$

De posse da altura $H(d)$ o **desempenho** do algoritmo sobre entrada d é:

- Se a entrada d satisfaz o teste *smpl*(d) (i. e. $\text{tam}(d) \leq 1$):
 $\text{desemp}[\text{Mergesort}](d) = \text{ava}[\text{tam}(d) \leq 1] + \text{desemp}[r \leftarrow d].$

- Se a entrada d não satisfaz o teste $\text{simpl}(d)$ (i. e. $\text{tam}(d) > 1$):

$$\begin{aligned} \text{desemp}[\text{Mergesort}](d) = & \text{aval}[\text{tam}(d) \leq 1] + \\ & + \text{desemp}[d_1 \leftarrow \text{Prim}(d)] + \text{desemp}[d_2 \leftarrow \text{Fin}(d)] + \\ & + \text{desemp}[\text{Mergesort}](\text{Prim}(d)) + \text{desemp}[\text{Mergesort}](\text{Fin}(d)) + \\ & + \text{desemp}[\text{Intercl}](\text{Mergesort}(\text{Prim}(d)), \text{Mergesort}(\text{Fin}(d))). \end{aligned}$$

Nesse caso, é natural tomar como **operação fundamental** a comparação entre elementos de Elmord.

Mas só $\text{cmbn_2}(r_1, r_2) := \text{Intercl}(r_1, r_2)$ envolve tais comparações.

O desempenho de $\text{Intercl}(r_1, r_2)$ é dado por

$$\text{desemp}[\text{Intercl}](r_1, r_2) = \text{tam}(r_1) + \text{tam}(r_2) - 1.$$

Então, em termos de número de comparações, obtemos as expressões:

Para a entrada **simples** (com $\text{tam}(d) \leq 1$):

$$\begin{aligned} \text{desemp}[\text{Mergesort}](d) = & \text{aval}[\text{tam}(d) \leq 1] + \text{desemp}[r \leftarrow d] = \\ & = 0 + 0 = 0. \end{aligned}$$

Para a entrada **não simples** (com $tam(d) > 1$):

$$\begin{aligned}
 desemp[Mergesort](d) &= aval[tam(d) \leq 1] + \\
 &+ desemp[d_1 \leftarrow Prim(d)] + desemp[d_2 \leftarrow Fin(d)] + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ desemp[Intercl](Mergesort(Prim(d)), Mergesort(Fin(d))) = \\
 &= 0 + 0 + 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam[Mergesort(Prim(d))] + tam[Mergesort(Fin(d))] - 1.
 \end{aligned}$$

Assim, já que Mergesort preserva tamanho: $tam(Mergesort(d)) = tam(d)$, temos

$$\begin{aligned}
 desemp[Mergesort](d) &= 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam(Prim(d)) + tam(Fin(d)) - 1.
 \end{aligned}$$

Uma vez que $tam(Prim(d)) + tam(Fin(d)) = tam(d)$, obtemos

$$\begin{aligned}
 desemp[Mergesort](d) &= 0 + \\
 &+ desemp[Mergesort](Prim(d)) + desemp[Mergesort](Fin(d)) + \\
 &+ tam(d) - 1.
 \end{aligned}$$

Assim, o desempenho do algoritmo Mergesort sobre entrada d é descrito pela recorrência

$$\text{desemp[Mergesort]}(d) = \begin{cases} 0 & \text{se } \text{tam}(d) \leq 1 \\ \text{desemp[Mergesort]}(\text{Prim}(d)) + \\ \text{desemp[Mergesort]}(\text{Fin}(d)) + \text{tam}(d) - 1 & \text{se } \text{tam}(d) > 1 \end{cases}$$

Agora, consideremos o **pior caso**, para entradas com tamanho até $n > 1$.

O tamanho máximo das decomposições de tais entradas vem a ser

$$s(n) := \text{Máx}\{S(d) \in \mathbb{N} / \text{tam}(d) \leq n\} \text{ e } s(n) = \lceil n/2 \rceil, \text{ logo } s(n) = \Theta(n/2).$$

A altura máxima das árvore de execução de tais entradas vem a ser

$$h^{\leq}(n) := \text{Máx}\{H(d) \in \mathbb{N} / \text{tam}(d) \leq n\} \text{ e } h^{\leq}(n) = \lceil \log_2 n \rceil, \text{ logo } h^{\leq}(n) = \Theta(\log_2 n).$$

Como $\text{part}_1(d) := \text{Prim}(d)$ e $\text{part}_2(d) := \text{Fin}(d)$, para $j = 1, 2$, temos

$$\text{desemp[Mergesort]}(\text{part}_j(d)) \leq c_{\bar{p}}^{\leq}[\text{Mergesort}](\text{part}_j(d)) \leq c_{\bar{p}}^{\leq}[\text{Mergesort}](s(n)).$$

Assim, uma **cota superior** $CS^{\leq}(n)$ para a ordem da complexidade pessimista de Mergesort pode ser dada pela seguinte recorrência

$$CS^{\leq}(n) = \begin{cases} 0 & \text{se } n \leq 1 \\ n + 2 \cdot CS^{\leq}\left(\frac{n}{2}\right) & \text{se } n > 1 \end{cases}$$

Agora, essa cota superior $CS^{\leq}(n)$ é dada por:

$$\begin{aligned} CS^{\leq}(n) &= n + 2 \cdot CS^{\leq}(n/2) = \\ &= n + 2 \cdot (n/2 + 2 \cdot CS^{\leq}(n/2^2)) = \\ &= n + n + 2^2 \cdot (n/2^2 + 2 \cdot CS^{\leq}(n/2^3)) = \\ &\quad \dots \\ &= n + n + \dots + n + \dots + 2^{h^{\leq}(n)} \cdot CS^{\leq}(n/2^{h^{\leq}(n)}). \end{aligned}$$

Então, o valor da cota superior $CS^{\leq}(n)$ é dado pela somatória:

$$\frac{n + \dots + n}{h^{\leq}(n)} + 0$$

Assim, obtemos $CS^{\leq}(n) = n \cdot h^{\leq}(n) = n \cdot \lceil \log_2 n \rceil$, logo

$$CS^{\leq}(n) = \Theta(n \cdot \log_2 n).$$

Portanto, para a complexidade pessimista do algoritmo Mergesort temos

$$c_p^{\leq}[\text{Mergesort}](n) \leq CS^{\leq}(n) = O(n \cdot \log_2 n).$$

Essa análise mostra que a complexidade pessimista do algoritmo Mergesort, em termos de número de comparações, tem ordem

$$c_p^{\leq}[\text{Mergesort}](n) = O(n \cdot \log_2 n).$$